

Week 4: SVMs and Backpropagation

<http://mlvu.github.io>

February 27, 2023

1 Support vector machines

We will practice a little with the maximum margin loss/support vector machine to ensure that you understand the details.

question 1: Jack says that support vector machines are a way to learn non-linear functions, that's what they're famous for. They use the kernel trick to do this. Jill says that Jack is wrong: support vector machines use a *linear* classifier trained by the maximum-margin loss function. Who, if anybody, is right?

In the linear case, the (soft margin) SVM finds a hyperplane, expressed as $\mathbf{w}^T \mathbf{x} + \mathbf{b}$, by solving the following optimization problem:

$$\begin{aligned} &\text{minimize } \frac{1}{2} \|\mathbf{w}\| + C \sum_i p_i \\ &\text{such that } y_i (\mathbf{w}^T \mathbf{x}_i + \mathbf{b}) \geq 1 - p_i \\ &\text{and } p_i \geq 0 \end{aligned}$$

question 2: What does i index? How many terms does the sum have, and how many constraints are there?

question 3: What is the value of y_i in this expression? What is its function?

After training, the class of an instance $\mathbf{x}$ is predicted based on the *sign* of $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + \mathbf{b}$. If $f(\mathbf{x})$ is negative, then the class is **negative**, and if it is positive, then the class is **positive**.¹

¹SVMs are only defined for binary classification problems. In binary classification, we

question 4: After training a linear SVM, we get $\mathbf{w} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}$ and $b = -1$. Is $\mathbf{x}_i = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$ with $y_i = 1$ is a support vector?

question 5: Does this mean that we expect the resulting classifier to satisfy $f(\mathbf{x}_i) = y_i$ for every instance, as in a regression problem?

question 6: Having resolved their previous disagreement Jack and Jill are now a little confused about what the “support vectors” are. They agree that they are a subset of the instances in the data. Jack thinks that they are instances for which $f(\mathbf{x}_i)$ is exactly equal to y_i . Jill thinks that they are the instances that are exactly on the margin defined by the classifier. Who, if anybody, is right?

For the remainder, we will use the following classifier:

$$\mathbf{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, b = 1$$

question 7: What classes are assigned to the points $\begin{pmatrix} 2 \\ 1 \end{pmatrix}$, $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$ and $\begin{pmatrix} 2 \\ 2 \end{pmatrix}$.

question 8: Draw the decision boundary in feature space, and label the points from the previous question. Label which side of the decision boundary is **positive** and which is **negative**. *Tip: what is the value of $\mathbf{w}^T \mathbf{x} + b$ when $\mathbf{x}$ is exactly on the decision boundary? Write this as a scalar equation, and rewrite to get the familiar representation of a line.*

question 9: Which of the following pairs are support vectors?

commonly call one class **positive** and the other **negative**. For multiclass problems, it's common to train one SVM per class to distinguish the class from the rest, and to combine these into a single classifier.

$$\mathbf{x}_1 = \begin{pmatrix} 1 \\ 3/2 \end{pmatrix}, y_1 = 1$$

$$\mathbf{x}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, y_2 = -1$$

$$\mathbf{x}_3 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, y_3 = 1$$

$$\mathbf{x}_4 = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}, y_4 = 1$$

question 10: Draw the margins of the classifier in the previous diagram, together with the points from the previous question. *Tip: the edges of the margins are defined by an equation, very similar to that of the decision boundary.*

question 11: To finish up with, here's a bit of puzzle. You don't have to be able to do this for the exam, but trying to solve it should help you to understand the finer details of how the parameters of the SVM determine the margin and the decision boundary.

The slides say that there are infinitely many classifiers with the same decision boundary, but that they all have different margins. Find a classifier that has the same decision boundary as the one we've been using so far, but whose margin is twice as wide.

2 Backpropagation

We will practice the backpropagation algorithm as described in the slides. We will use a neural network defined by the following function:

$$y = v_1 h_1 + v_2 h_2$$

$$h_1 = \sigma(k_1)$$

$$h_2 = \sigma(k_2)$$

$$k_1 = w_1 x$$

$$k_2 = w_2 x$$

Where σ represents the logistic sigmoid. The network has a single input node x and a single output node y . The weights are w_1 , w_2 , v_1 and v_2 . We've left out bias nodes to keep things simple.

question 12: Draw this network.

We will use stochastic gradient descent to train this network. That means we define the loss function for a single instance. We will use basic least-squares loss to train this network for regression. Our loss function for instance x is

$$\text{loss}_x(w_1, w_2, v_1, v_2) = \frac{1}{2} (y - t)^2$$

where t is the target value provided by the dataset, and y is the output of the network.

We see each line in the definition above as a module, with some inputs and some outputs. Using the chain rule, we will express the derivative with respect to weight w_1 . We will use only *local derivatives*, expressing the derivative for each module with respect to its inputs, but not working out the derivative beyond that.

question 13: Fill in the gaps:

$$\begin{aligned} \frac{\partial \text{loss}}{\partial w_1} &= \frac{\partial \text{loss}}{\partial y} \frac{\partial y}{\partial h_1} \cdots \frac{\partial k_1}{\partial w_1} \\ &= (y - t) \times \dots \times \sigma(k_1)(1 - \sigma(k_1)) \times \dots \end{aligned} \quad (1)$$

question 14: The network has a diamond shape, just as shown in the slide when explaining the multivariate chain rule (in the *Deep Learning 1* lecture). However, in this case, we don't need the multivariate chain rule. Why not? In what kind of situation would the multivariate chain rule be required?

We could take the formulation above and fill in the symbolic expressions for y , k_1 , etc, and come up with a general symbolic formula for the derivative for all inputs. However, that is usually expensive to do for large neural nets. Instead, we leave it as is, and fill in the *numeric* values for these variables for a specific input and for specific weights.

question 15: Assume that the input is $x = 1$, with target output $t = \frac{1}{2}$ and that all weights are set at 1. Do a *forward pass*: compute the loss and all intermediate values k_1 , k_2 , h_1 , h_2 and y .

To simplify the calculation, you can use the approximation $\sigma(1) = \frac{3}{4}$.

Now we do the *backward pass*. Fill these intermediate values in to the loss function decomposed by the chain rule from line 1, and compute the derivative with respect to w_1 for these inputs and weights.

3 Bonus: Normalization

This section was part of an earlier version of the homework. It doesn't correspond to an application question, so you can skip this if you like, but if you want to practice the basis transformations from the normalization video, you can use these exercises.

In this section we'll look at the theory behind whitening. The idea of a *basis transformation* is important to understand, so we'll start there.

Basis systems The *standard basis* (in 2 dimensions) is just the vectors $b_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $b_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. When a point has coordinates (x, y) , that just means that we can get to the point by adding x times the b_1 vector to y times the b_2 vector.

For the standard basis, we just end up at the point $\begin{pmatrix} x \\ y \end{pmatrix}$. But we can also use a nonstandard basis. If a point p has coordinates (x, y) in the coordinate system with basis vectors b_1, b_2 , it means that p has coordinates $xb_1 + yb_2$ in the standard basis system.

question 16: Convert the following points to a representation in the standard basis system.

1. The point $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ expressed in the coordinate system with basis vectors $b_1 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, b_2 = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$.
2. The point $\begin{pmatrix} 5 \\ 3 \end{pmatrix}$ expressed in the coordinate system with basis vectors $b_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, b_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$.

3. The point $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$ expressed in the coordinate system with basis vectors $b_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, b_2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$.

question 17: A basis system is **orthonormal** if two conditions hold:

1. ...
2. ...

Consider the following three bases:

$$\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \quad \mathbf{B} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \quad \mathbf{C} = \sqrt{1/2} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

The columns of each matrix are the basis vectors (it may help to draw them in standard coordinates). Which of these have orthogonal basis vectors? Which one is orthonormal?

question 18: Why is it useful to have an orthonormal basis?

Covariance and normal distributions PCA is closely related to the normal distribution. We can think of PCA as a basis transformation that makes it look like the data was sampled from a standard multivariate normal distribution: it makes the variance 1 in every dimension, and the covariance 0 for every pair of dimensions.

question 19: Here is some data.

x_1	x_2
0	2
2	0
0	2
2	0

Compute the sample covariance.

Let $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{t}$ be an *affine* transformation (a linear transformation $\mathbf{A}$ followed by a translation $\mathbf{t}$). We draw $\mathbf{x}$ from a standard normal distribution

$$\mathbf{N}(\mathbf{0}, \mathbf{I}) = \mathbf{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right)$$

and compute $\mathbf{y}$. This is equivalent to sampling from a normal distribution with mean $\boldsymbol{\mu} = \mathbf{t}$ and covariance $\boldsymbol{\Sigma} = \mathbf{A}\mathbf{A}^\top$.

question 20: Let

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \mathbf{t} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Sampling $\mathbf{x}$ from $N(\mathbf{0}, \mathbf{I})$ is easy: we just sample x_1 and x_2 independently from a *univariate* standard normal distributions. Here are some **standard normally distributed numbers**:

$$0.5, 1.1, 0.1, 0.9, -0.2, 1.3$$

Turn these into three samples from $N(\mathbf{0}, \mathbf{I})$ and transform them to samples from $N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

Bonus question: Compute the sample covariance of the resulting data, and the actual covariance $\boldsymbol{\Sigma}$.